

KLEPSYDRA AI FOR THE GR740: GUIDE FOR THE DEPLOYMENT ON LINUX AND RTEMS V25.5.5

REV 29.08.2026

ANNEX A: Klepsydra AI on the GR740 on RTEMS	4
Overview	4
Prerequisites	4
Deployment procedure	4
Benefits of the solution	4
Installation package content and requirements	6
Prerequisites and Requirements	6
Software Package Content	6
Example deployment guide	7
Step 0: Download and installation of the software	8
Download Klepsydra AI software for GR740	8
Start the docker	9
Run GR740 Cross Compilation docker container.	9
Step 1: Download and preparation of an example model	10
Download an open source model	10
Model Simplification (inside or outside the docker container)	10
Convert the onnx to kpsr	11
Convert the kpsr to c (Serialise the KPSR model to a C array)	11
Step 2: Compilation of the AI application	13
Compile an example + model + libraries	13
Step 3: Deployment	15
Run on GR740 using GRMON.	15
ANNEX B: Klepsydra AI RTEMS Example Explained	17
Main Application file	17
Header (e.g. ai-rtems-example/main.cpp)	17
Klepsydra AI Configuration	18
Klepsydra AI Inference creation and model loading	18
Execution	19
Performance Benchmarks	19
ANNEX C: Klepsydra AI on the GR740 on Linux	20
Overview	20
Prerequisites	20
Deployment procedure	20
Benefits of the solution	21
Installation package content and requirements	22
Prerequisites and Requirements	22
Software Package Content	22
Example deployment guide	23

Step 0: Software installation	24
Download Klepsydra AI software for GR740	24
Start the docker	24
Run GR740 Cross Compilation docker container.	24
Step 1: Preparation of the Linux Image with Klepsydra AI binaries	25
Overview	25
Building and Deploying the Linux Image	25
Validation of the Linux Image	26
Step 2: Compilation of the AI examples	27
Overview	27
Cross-compile the binaries within the Docker container environment	27
Preparation of the image data for the example ai-min-raw-examples	28
Packaging	28
Step 3: Deployment and execution	29
Copy the compiled example tarballs to the GR740	29
Pre-process the images for the ai-min-raw-images-example	29
Execution	30
Troubleshooting	31
Licensing issues	31
Connectivity issues	32
About Klepsydra	33
Overview	33
Contact	33

ANNEX A: Klepsydra AI on the GR740 on RTEMS

Overview

This annex details the procedure for deploying Artificial Intelligence (AI) applications onto the GR740 processor, utilizing the Klepsydra AI libraries specifically compiled for the RTEMS 6 Symmetric Multi-Processing (SMP) environment. The targeted deployment architecture is illustrated in the diagram below:

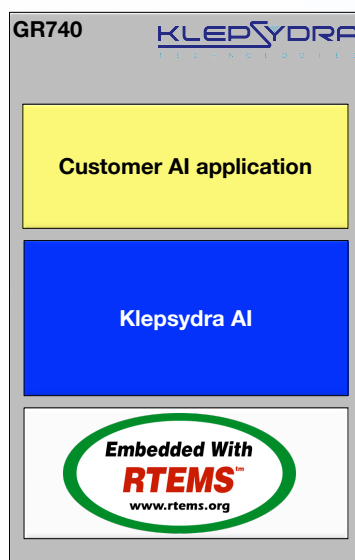


Figure 1: GR740 Software Stack

Prerequisites

This document is focused strictly on the deployment procedure and does not cover the general configuration, initialisation, or full Application Programming Interface (API) of the Klepsydra framework.

Before proceeding with this annex, the user must be familiar with and consult two guides:

- Klepsydra AI Getting Started Guide v25.5.4
- Klepsydra AI ONNX Loader V25.5.4

Deployment procedure

Due to the lack of an inherent filesystem in the RTEMS 6 operating environment, all necessary components for execution must be statically integrated into a single deployable binary image. This monolithic binary will contain the following required elements (as shown in the figure 2 below):

- The Klepsydra AI core libraries.
- The customer's application code.
- The compiled neural network algorithm file.
- All associated Klepsydra AI configuration files.

The remainder of this document details the specific procedures required to create, build, and deploy this composite binary image onto the GR740 target processor.

Benefits of the solution

This integrated deployment methodology, leveraging Klepsydra AI on the GR740/RTEMS 6 SMP platform, delivers significant operational and performance advantages:

- **Accelerated Deployment and Performance:** The solution enables the rapid deployment of AI applications with performance optimized specifically for the GR740 architecture.
- **Multicore Execution:** The utilization of RTEMS 6 SMP inherently supports the execution of AI applications across multiple cores, maximizing the available processing power of the GR740.
- **Superior Performance Benchmarks:** Compared to established alternatives like TensorFlow Lite Micro and Apache TVM, Klepsydra AI consistently demonstrates substantially better processing efficiency on the GR740:
 - Achieving up to 2× faster processing in single-core execution.
 - Delivering 6× to 8× faster processing during multicore execution.

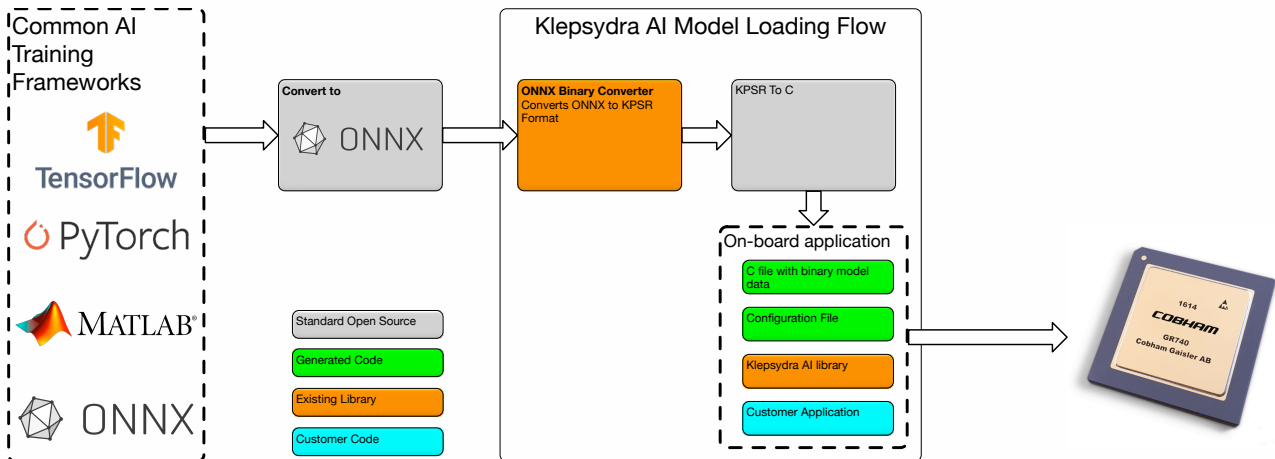


Figure 2: Deployment workflow including the integrated binary with model, AI libraries, application code and configuration files.

Installation package content and requirements

Prerequisites and Requirements

To successfully follow the procedures outlined in this guide, the following elements are required:

Category	Requirement	Notes
Development Host	Linux-based Laptop or PC	With Docker support.
Software	Docker Engine	Required for running the provided development environment image.
Target Access	Connection to GR740 Hardware or TSIM Simulator	Necessary for uploading and executing the final deployable binary.
Knowledge	Makefile, Docker and Linux Command Line	Fundamental understanding of these environments is assumed.

Software Package Content

The provided software delivery consists of a comprehensive Docker image containing all necessary components to streamline the development and deployment process:

- **Klepsydra AI GR740 Binaries:** Pre-compiled libraries and tools required for the GR740/RTEMS 6 SMP target.
- **Reference Examples:** A complete set of examples, including fully functional customer application code, associated Klepsydra configuration files, and a sample AI model.
GR740 RTEMS Version: Only contains AI examples for RTEMS and DNN Statistic example.
- **Cross-Compilation Toolchain:** The necessary tools for building the deployable binary. Note: The specific version of the SPARC/LEON cross-compilation toolchain may be upgraded or downgraded according to the customer's existing licensing agreement with Gaisler. The main requirements are to have GCC 8 or above, atomic support and RTEMS6 SMP support.
- **Klepsydra AI Docker Binaries and Tools:** A suite of conversion tools for model preparation and cross-platform deployment. It includes utilities to convert ONNX models into the Klepsydra (.kpsr) format, as well as data transformation tools to map numeric formats between x86_64/AArch64 host environments and the GR740 target architecture.

The Klepsydra AI Docker image is structured to provide all necessary binaries, header files, examples, and tools in standard, easily accessible locations within the container environment.

Specifically, the key contents are located as follows:

Directory/File	Content Description
/usr/local/lib	Binaries (Libraries and executables).
/usr/local/include	Header Files (C++ interface files).
/usr/local/src	Example Code (Source files for demonstration projects).
/usr/local/share/doc/kpe-ai/ Makefile.sample	A Sample Makefile provided for reference and configuration of build processes.
/opt/rtems/6/bin/sparc-rtems6-gcc	Cross-Compiler (GCC) for the SPARC/RTEMS 6 target architecture.
/opt/rtems/6/bin/sparc-rtems6-g++	Cross-Compiler (G++) for the SPARC/RTEMS 6 target architecture.
/usr/local/bin/ kpsr_ai_onnx_compatible_checker_app	ONNX Compatibility Checker utility (runs on X84, 64Bit Linux only)
/usr/local/bin/kpsr_ai_onnx_converter	ONNX Model Converter tool.

This structure ensures all necessary components for development, cross-compilation, and ONNX model preparation are readily available within the Docker environment.

Example deployment guide

This section provides a step-by-step procedural guide designed to walk the user through the process of compiling, deploying, and executing a complete Klepsydra AI application example on the GR740 target hardware. There are four sections corresponding to the four parts of the process:

- Step 0: Download and installation of the software
- Step 1: Download and preparation of an example model
- Step 2: Compilation of the AI application
- Step 3: Deployment.

Step 0: Download and installation of the software

Download Klepsydra AI software for GR740

Download Klepsydra AI software for GR740

Please NOTE:

The download page is currently being updated and is temporarily unavailable. In the meantime, software packages are being provided directly via S3 links provided via email. Thus the following sections can be ignored and please go to "[Start the docker](#)" further below to continue the installation.

- Navigate to <https://downloads.klepsydra.com/>
- Login with the provided credentials.



Klepsydra Download Page

Sign in

A sign-in form with a text input field labeled 'Email', a blue 'Continue' button, and a link 'or' followed by a 'Sign in with a passkey' button. A link 'Don't have an account?' is located below the buttons.

Email

Continue

or

Sign in with a passkey

[Don't have an account?](#)

Figure 3: Download page log-in page

The system supports the following methods for user authentication:

- Password: Standard authentication using a user-defined alphanumeric string.
- Passcode: Authentication via a short, typically numeric, time-sensitive code (e.g., used in Multi-Factor Authentication).
- Passkey Generation: Allows the user to create a cryptographic credential (Passkey) for secure, passwordless login using local device mechanisms (e.g., biometrics).

Create account

A form for creating an account with a text input field labeled 'Email' and a blue 'Continue' button. A link 'Already have an account?' is located below the button.

Email

Continue

[Already have an account?](#)

Figure 4: Login page

At the download page:

- Select RTEMS AI section
- Select processor, distribution and format type options

- Click on “Download .tag.gz”

Start the docker

Load docker image tarball:

```
$ docker load < kpe-ai-rtems6-gr740-dev_docker_25.5.4.tar.gz
```

```
Loaded image: kpe-ai-rtems6-gr740-dev:25.5.4
```

Run GR740 Cross Compilation docker container.

Run container in detached mode¹:

```
$ docker run -itd --name CONTAINER_NAME klepsydra-technologies/kpe-ai-rtems6-gr740-dev:25.5.4
```

Check that the container is running:

```
$ docker ps
```

To interact with the running container environment, you must attach to its execution session using either the container name or the container ID:

```
$ docker exec -it CONTAINER_NAME bash
```

```
root@<CONTAINER_ID>:/#
```

To exit from container execution just type “exit”

¹ Where CONTAINER_NAME is any valid text, e.g., klepsydra_ai_gr740

Step 1: Download and preparation of an example model

This section details the necessary actions to transform an ONNX file defining the neural network into a C source file. The result is a model represented as a string variable that can be statically included within the final GR740 / RTEMS 6 application binary, serialised as a C array statically build.

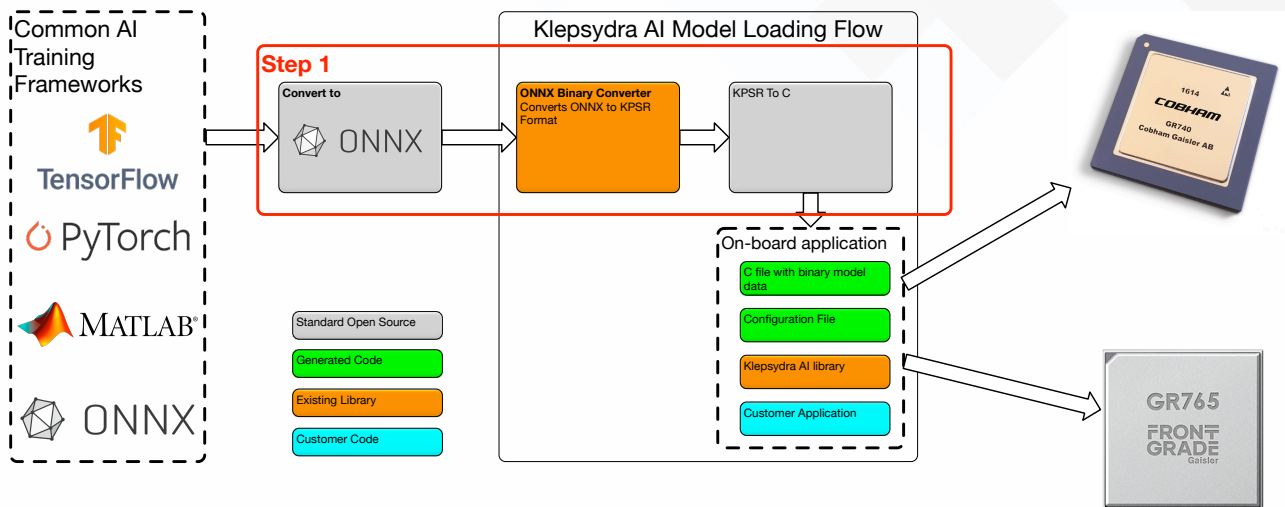


Figure 6: Step 1 of the AI application deployment process

Download an open source model

The procedure outlined in this guide utilises AI models in the ONNX format. Klepsydra AI supports ONNX models up to opset version 12 and requires the channel-last data format.

The deployment steps are universally applicable to any compatible ONNX model. For this walkthrough, we will download and use the example model `mobilenetv2-12.onnx`² from the ONNX Model Repository:

<https://github.com/onnx/models/tree/main/validated/vision/classification/mobilenet>

Model Simplification (inside or outside the docker container)

It is highly recommended to use the ONNX Simplifier tool (available at: <https://github.com/daquexian/onnx-simplifier>) to optimize the ONNX graph³. This tool eliminates all the unnecessary artificial layers created during the ONNX export process from TensorFlow or PyTorch. Running this tool, ensure compatibility with Klepsydra AI as well as cleaner models.

Action:

```
$ onnxsim --overwrite-input-shape="1,3,224,224" mobilenetv2-12.onnx mobilenetv2-simplified.onnx
```

For detailed assistance concerning the `value_info` field within the ONNX model specification, please consult the following external documentation:

<https://onnx.ai/onnx/repo-docs/IR.html#graphs>

https://onnx.ai/onnx/api/helper.html#onnx.helper.make_tensor_type_proto

² This model accepts 3 channel inputs of any dimensions. We define the input to have the same size as the model.

³ If the network contains quantized layers that lack explicit output shape information, the simplification process may not execute efficiently. Contact support@klepsydra.com if issues arise during this process on quantised models.

Simplifying...
Finish! Here is the difference:

	Original Model	Simplified Model
Add	10	10
Clip	35	35
Concat	1	0
Constant	178	109
Conv	52	52
Gather	1	0
Gemm	1	1
GlobalAveragePool	1	1
Reshape	1	1
Shape	1	0
Unsqueeze	1	0
Model Size	13.3MiB	13.3MiB

Figure 7: Simplification tool output

For detailed information regarding Klepsydra DNN layer compatibility and the procedure for exporting trained models to ONNX (from frameworks like TensorFlow or PyTorch), please consult the dedicated guide Klepsydra “AI ONNX Loader” as well as the support matrix and “Getting Started AI guide”.

Convert the onnx to kpsr

The required ONNX conversion utility, `kpsr_ai_onnx_converter`, is pre-installed and available directly within the provided Docker container.

Run Conversion Tool: Execute the `kpsr_ai_onnx_converter` utility on the copied model file.

```
# kpsr_ai_onnx_converter -m mobilenetv2-simplified.onnx -o dnn_model.kpsr
```

```
Input file provided:  mobilenetv2.onnx
Output filename :    mobilenetv2.kpsr
[2025-07-23 17:39:09.413] [info] Reading onnx data from file mobilenetv2.onnx
[2025-07-23 17:39:09.413] [info] Converting network from onnx format to Klepsydra format
[2025-07-23 17:39:09.424] [info] Network expects input of shape [batch = 1, 224 x 224 x 3] (with channels as the innermost)
[2025-07-23 17:39:09.452] [info] Network conversion successful. Starting serialization to disk
[2025-07-23 17:39:09.453] [info] Converted network expects input of size 224 x 224 x 3
[2025-07-23 17:39:09.463] [info] Network serialized to Klespydra binary format
[2025-07-23 17:39:09.463] [info] Network successfully saved to file mobilenetv2.kpsr
```

Figure 8: Output of the conversion tool

Convert the kpsr to c (Serialise the KPSR model to a C array)

The container also includes a complete Klepsydra example project with all required source files. To execute the converted model generated in the previous steps, you must now place the `dnn_model.c` file inside the example project folder.⁴

```
$ cp dnn_model.kpsr /usr/local/src/kpe-ai-examples/ai-rtems-example/
$ cd /usr/local/src/kpe-ai-examples/ai-rtems-example/
```

⁴ The following commands require “sudo” permission. In case you prefer not to invoke “sudo”, use “cp -a” into a writable folder

The final step in model preparation is to serialize the converted Klepsydra model into a C source file. This allows static integration into the final deployable binary. Use the standard `xxd` utility for this purpose.

```
$ xxd -i ./dnn_model.kpsr > src/dnn_model.c
```

Be aware that not all converted model sizes are compatible with the target memory capacity of the GR740 processor. Exceeding this capacity will lead to memory allocation failures during runtime execution.

Step 2: Compilation of the AI application

This section outlines the actions required to successfully cross-compile and generate the deployable image file for the GR740 / RTEMS 6 target environment.

The provided example utilises a Makefile-based compilation approach. Users are free to adapt these principles or substitute them with an alternative build methodology that aligns with their established building processes.

While the provided Docker image includes a validated toolchain, users are free to utilise alternative cross-compilation approaches and/or different versions of the compilation tools. This flexibility is permitted provided the chosen tools maintain full support for the GR740 processor and the RTEMS 6 SMP environment.

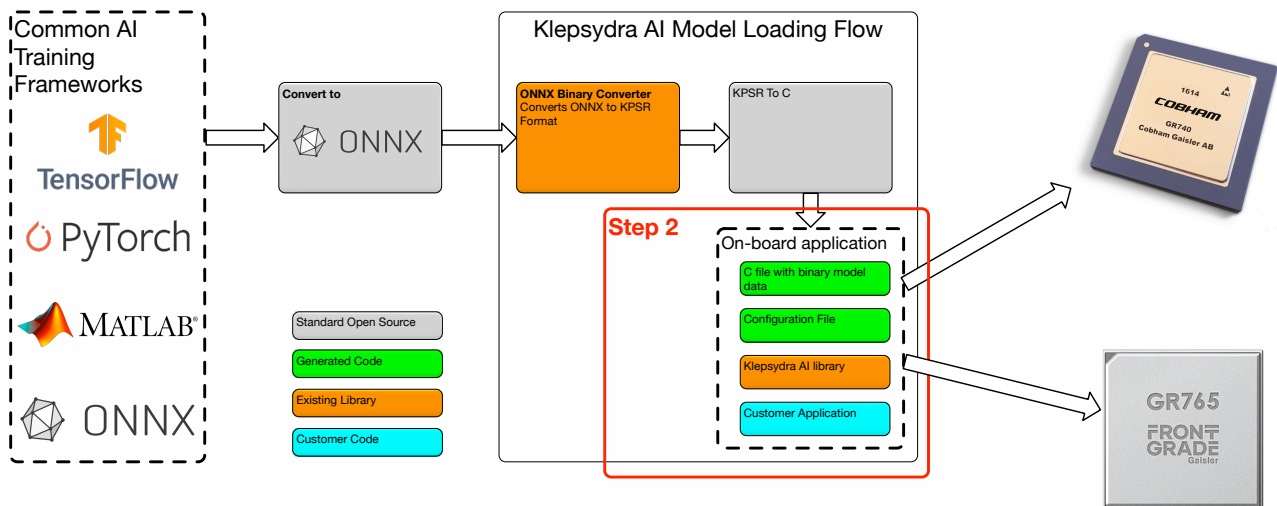


Figure 9: Step 2 of the AI application deployment process

Compile an example + model + libraries

The running Docker container is pre-configured with the necessary Klepsydra cross-compilation toolchain and libraries for the GR740 target⁵.

Copy the Makefile sample file into the Klepsydra example directory.⁶

```
# cd /usr/local/src/kpe-ai-examples/ai-rtems-example/
# cp /usr/local/share/doc/kpe-ai/Makefile.sample Makefile
```

Execute the make command to compile the project.

```
# make
```

Expected output:

```
/opt/rtems/6/bin/sparc-rtems6-g++ -mcpu=leon3 -isystem/opt/rtems/6/sparc-rtems6/gr740/lib/include src/dnn_model.o src/main.o -o example_program -L/usr/local/lib -B/opt/rtems/6/sparc-rtems6/gr740/lib -qrtems -Wl,--gc-sections -lkpe_ai -lrtemsbasp -lrtemsdefaultconfig -lrtemscpu -lbsd
```

The successful compilation will generate the final deployable binary with the name: `example_program`.

⁵ The folder already contains a generic model for testing purposes.

⁶ Requires “sudo” permission or copy instead to a writable folder

```
root@d249b7f14682:/usr/local/src/kpe-ai-examples/ai-rtems-example# ll
total 17696
drwxr-xr-x 1 1000 1000      4096 Jul 23 14:34 ./
drwxr-xr-x 1 1000 1000      4096 May 15 17:18 ../
-rwxr-xr-x 1 root  root 18174408 Jul 23 14:34 example_program*
-rw-r--r-- 1 root  root   1910 Jul 23 14:33 Makefile
drwxr-xr-x 1 1000 1000      4096 Jul 23 14:06 src/
```

Figure 10: Content of the example folder after compilation.

Once compilation is complete, download the generated binary from the container to your host machine.

Use the following command, ensuring you replace <CONTAINER_NAME_or_name> with the correct container identifier

```
# docker cp <CONTAINER_NAME_or_name>:/usr/local/src/kpe-ai-examples/ai-rtems-example/example_program .
```

Step 3: Deployment

This section covers the final stage of the process: deployment and execution on the target environment. To successfully complete this stage, access to either the GR740 hardware platform or the TSIM simulator is mandatory.

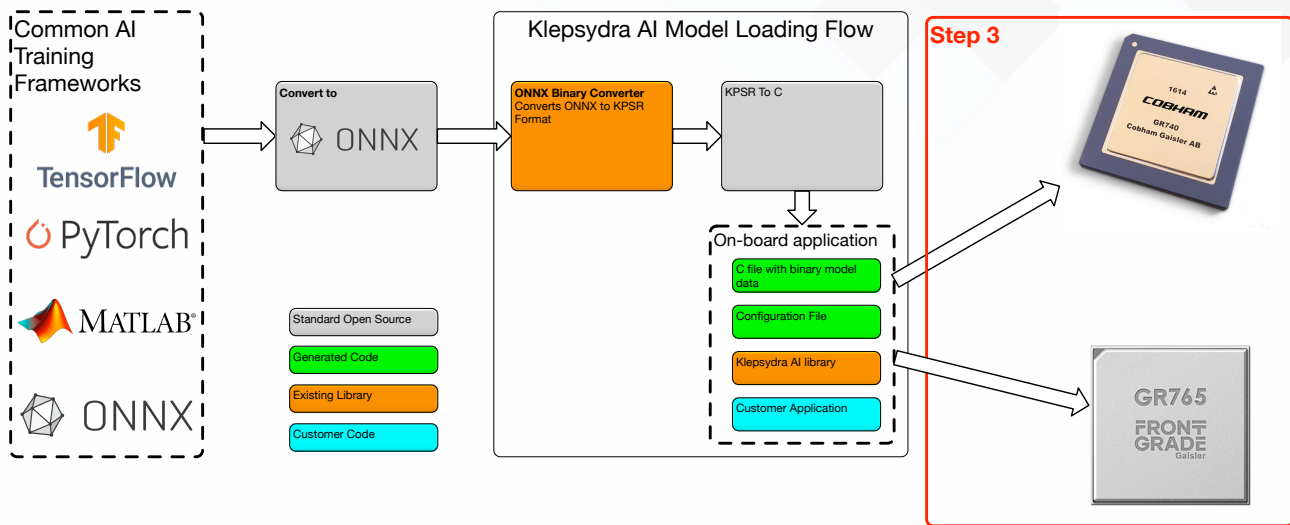


Figure 11: Step 3 in the AI application deployment process.

Run on GR740 using GRMON.

This step details how to load and execute the generated `example_program` binary on the GR740 target, utilising either the physical hardware or the TSIM simulator.

Copy the `example_program` binary to a server that has network access to the physical GR740 board (e.g., using Secure Copy Protocol - SCP):

Remark: Example copy command. Please adapt to your local setup

```
$ scp example_program klepsydra@r0.gaisler.com:/home/klepsydra/
```

Establish an SSH session with the transfer server:

Remark: Example copy command. Please adapt to your local setup

```
$ ssh klepsydra@r0.gaisler.com
```

From the directory containing the binary, start the GRMON3 debugging tool, targeting the physical GR740 hardware, enabling JTAG/FTDI connection, and forwarding console output:

```
$ sudo LD_LIBRARY_PATH=/opt/grmon-eval-4.1.2/linux/lib64/ /opt/grmon-eval-4.1.2/linux/bin64/grmon -ftdi -sddcs -nb -jtagserial GR740M0040A -u -e "load /home/klepsydra/example_program; run"
```

The expected output should look like this:

```
sudo LD_LIBRARY_PATH=/opt/grmon-eval-4.1.2/linux/lib64/ /opt/grmon-eval-4.1.2/linux/bin64/grmon -ftdi -sddcs -nb -jtagserial GR740M0040A -u -e "load /home/klepsydra/example_program; run"
```

```
GRMON debug monitor v4.1.2 64-bit eval version
```

```
Copyright (C) 2026 Frontgrade Gaisler - All rights reserved.
For latest updates, go to https://www.gaisler.com/
Comments or bug-reports to support@gaisler.com
```

```
This eval version will expire on 25/12/2026
```

```
JTAG chain (1): GR740
```

Device ID: 0x740
 GRLIB build version: 4153
 Detected system: GR740 rev1
 Detected frequency: 250.0 MHz

Component	Vendor
JTAG Debug Link	Frontgrade Gaisler
GRSPW2 SpaceWire Serial Link	Frontgrade Gaisler
EDCL master interface	Frontgrade Gaisler
EDCL master interface	Frontgrade Gaisler
LEON4 SPARC V8 Processor	Frontgrade Gaisler
LEON4 SPARC V8 Processor	Frontgrade Gaisler
LEON4 SPARC V8 Processor	Frontgrade Gaisler
LEON4 SPARC V8 Processor	Frontgrade Gaisler
IO Memory Management Unit	Frontgrade Gaisler
AHB-to-AHB Bridge	Frontgrade Gaisler
L2-Cache Controller	Frontgrade Gaisler
AHB Memory Scrubber	Frontgrade Gaisler
IOMMU secondary master i/f	Frontgrade Gaisler
AHB-to-AHB Bridge	Frontgrade Gaisler
LEON4 Debug Support Unit	Frontgrade Gaisler
AHB/APB Bridge	Frontgrade Gaisler
AMBA Trace Buffer	Frontgrade Gaisler
AHB/APB Bridge	Frontgrade Gaisler
AHB/APB Bridge	Frontgrade Gaisler
Muxed FT DDR/SDRAM controller	Frontgrade Gaisler
Memory controller with EDAC	Frontgrade Gaisler
GRPCI2 PCI/AHB bridge	Frontgrade Gaisler
GRSPW Router	Frontgrade Gaisler
LEON4 Statistics Unit	Frontgrade Gaisler
GRPCI2 Trace buffer	Frontgrade Gaisler
Generic UART	Frontgrade Gaisler
Generic UART	Frontgrade Gaisler
General Purpose I/O port	Frontgrade Gaisler
Multi-processor Interrupt Ctrl.	Frontgrade Gaisler
Modular Timer Unit	Frontgrade Gaisler
Modular Timer Unit	Frontgrade Gaisler
Modular Timer Unit	Frontgrade Gaisler
Modular Timer Unit	Frontgrade Gaisler
Modular Timer Unit	Frontgrade Gaisler
GRSPW Router DMA interface	Frontgrade Gaisler
GRSPW Router DMA interface	Frontgrade Gaisler
GRSPW Router DMA interface	Frontgrade Gaisler
GRSPW Router DMA interface	Frontgrade Gaisler
GR Ethernet MAC	Frontgrade Gaisler
GR Ethernet MAC	Frontgrade Gaisler
CAN Controller with DMA	Frontgrade Gaisler
CAN Controller with DMA	Frontgrade Gaisler
SPI Controller	Frontgrade Gaisler
Clock gating unit	Frontgrade Gaisler
MIL-STD-1553B Interface	Frontgrade Gaisler
AHB Status Register	Frontgrade Gaisler
AHB Status Register	Frontgrade Gaisler
General Purpose I/O port	Frontgrade Gaisler
General Purpose Register	Frontgrade Gaisler
Temperature sensor	Frontgrade Gaisler
General Purpose Register Bank	Frontgrade Gaisler
CCSDS TDP / SpaceWire I/F	Frontgrade Gaisler
LEON4 Statistics Unit	Frontgrade Gaisler

Use command 'info sys' to print a detailed report of attached cores

0 .text	4.3MB /	4.3MB	[=====>]	100%
444430 .tdata	28B		[=====>]	100%
444450 .rtmsrset	2.1kB /	2.1kB	[=====>]	100%
44c2c0 .data	13.6MB /	13.6MB	[=====>]	100%
11db7c0 .rtmsrwset	1.7kB /	1.7kB	[=====>]	100%
11dbea8 .got	14.2kB /	14.2kB	[=====>]	100%

Total size: 17.84MB (774.98kbit/s)

Entry point 0x00000000

Image /home/klepsydra/example_program loaded

Example start

```
virtual bool kpsr::mem::SimpleMemEnv::getPropertyBool(const std::string&, bool&, bool, const std::string&): Key
'log_sdo_enable' not found. Using default value 'false'
virtual bool kpsr::mem::SimpleMemEnv::getPropertyString(const std::string&, std::string&, const std::string&,
const std::string&): Key 'log_filename' not found. Using default value ''
virtual bool kpsr::mem::SimpleMemEnv::getPropertyString(const std::string&, std::string&, const std::string&,
const std::string&): Key 'log_sdo_filename' not found. Using default value ''
virtual bool kpsr::mem::SimpleMemEnv::getPropertyBool(const std::string&, bool&, bool, const std::string&): Key
'export_through_tftp' not found. Using default value 'false'
virtual bool kpsr::mem::SimpleMemEnv::getPropertyString(const std::string&, std::string&, const std::string&,
const std::string&): Key 'container_name' not found. Using default value ''
virtual bool kpsr::mem::SimpleMemEnv::getPropertyString(const std::string&, std::string&, const std::string&,
const std::string&): Key 'streaming_configuration' not found. Using default value ''
virtual bool kpsr::mem::SimpleMemEnv::getPropertyString(const std::string&, std::string&, const std::string&,
const std::string&): Key 'container_name' not found. Using default value ''
```

ANNEX B: Klepsydra AI RTEMS Example Explained

This section examines the example code to highlight the key differences between the Klepsydra AI API for RTEMS 6/GR740 and its standard Linux counterpart.

The core API functionality remains fundamentally the same. The principal distinction is how input configuration and model data are handled: files are passed as static vector string objects (variables) instead of traditional filesystem file paths.

Users requiring detailed information about the overall Klepsydra API are strongly recommended to consult the main Klepsydra AI Getting Started Guide.

Main Application file

Header (e.g. ai-rtems-example/main.cpp)

This section details the RTEMS-specific environment setup within the application code, covering necessary configuration variables, required include files, and the main function declaration.

Note the two highlighted sections:

- Yellow: The RTEMS configuration variable required for multi-core AI execution—set this to your desired core count.
- Red: The forward declaration for the model binary string, generated during Step 1 (Model Conversion).

```
#ifdef __rtems__

#include <bsp.h>

/* Simple RTEMS configuration */
#define CONFIGURE_INIT TASK_ATTRIBUTES RTEMS_FLOATING_POINT
#define CONFIGURE_APPLICATION_NEEDS_CLOCK_DRIVER
#define CONFIGURE_APPLICATION_NEEDS_CONSOLE_DRIVER
#define CONFIGURE_UNIFIED_WORK_AREAS
#define CONFIGURE_UNLIMITED_ALLOCATION_SIZE 8
#define CONFIGURE_UNLIMITED_OBJECTS
#define CONFIGURE_RTEMS_INIT_TASKS_TABLE
#define CONFIGURE_INIT

/* Uncomment and change the number of max processors to test SMP for different
   hardware. If undefined, default value used is 1. */
// #define CONFIGURE_MAXIMUM_PROCESSORS 4

#include <rtems/confdefs.h>

#ifndef RTEMS_SMP
#ifndef RTEMS_DEOSAARCH64
#error "SMP support is disabled"
#endif /* RTEMS_DEOSAARCH64 */
#endif /* RTEMS_SMP */

#else

#include <unistd.h>

#endif /* __rtems__ */

#include <thread>

#include <klepsydra/ai/dnn_importer_with_environment.h>
#include <klepsydra/sdk/simple_mem_env.h>

#define TEST_ITERATIONS 10

extern unsigned char dnn_model_kpsr[];
extern unsigned int dnn_model_kpsr_len;

#ifdef __rtems__
void Init(rtems_task_argument arg)
#else
```

```
int main()
#endif
{
```

Klepsydra AI Configuration

The following section illustrates the procedure for programmatically configuring the Klepsydra AI application within the code⁷.

While it is also possible to provide configuration data as a static string object, this alternative method is outside the scope of this particular Getting Started Guide.

```
printf("Example start\n");

// Callback synchronization; Klepsydra libs are lock-free
std::atomic_bool ai_data_ready{};

// Configure environment
kpsr::mem::SimpleMemEnv environment;
environment.setPropertyBool("export_streaming_configuration", false);
environment.setPropertyBool("stat_file_container_enable", false);
environment.setPropertyBool("stat_socket_container_enable", false);
environment.setPropertyBool("test_mode", false);
environment.setPropertyBool("use_default_streaming_factory", true);
environment.setPropertyBool("use_policy_by_ratio", false);
environment.setPropertyInt("log_level", 2);
environment.setPropertyInt("non_critical_thread_pool_size", 1);
// For best performance number_of_cores should not exceed number of
// available cores. Set to 1 for best latency. Values > 1 may provide
// better throughput or CPU performance stats.
environment.setPropertyInt("number_of_cores", 1);
// critical thread pool size should be >= number_of_cores + 2.
environment.setPropertyInt("critical_thread_pool_size", 3);
// number_of_parallel_threads - intra threads used for parallelisation.
// Can be larger than CORES_NUMBER but should not be larger than actual available cores
// (physical or virtual).
environment.setPropertyInt("number_of_parallel_threads", 1);
environment.setPropertyInt("pool_size", 1);
```

To increase `number_of_parallel_pthreads`, you must first enable the `CONFIGURE_MAX_PROCESSORS` property in the `main.cpp` header. Otherwise, the following error will occur:

```
[Fri Jan 1 00:00:03 1988] [00:00:03 277237] [klepsydra] [info] [1848303606] CPUs for
parallel thread affinity [0, 1, 2, 3]
Failed to set rtems scheduler task [3]
Failed to set rtems scheduler task [3]
Failed to set rtems scheduler task [3]
```

Klepsydra AI Inference creation and model loading

The subsequent section demonstrates the creation of the Deep Neural Network (DNN) Factory and the instantiation of the DNN Inference object.

This code block is virtually identical to its Linux counterpart. The only difference is that the model data is supplied directly via the vector data string variable, as highlighted in red, instead of using a traditional filesystem path.

```
std::string model_name = "dnn_model";
std::vector<char> vec_dnn_model(dnn_model_kpsr, dnn_model_kpsr + dnn_model_kpsr_len);
auto factory = kpsr::ai::DnnImporterWithEnvironment::createDNNFactory(vec_dnn_model,
                                                                    model_name,
                                                                    &environment);

auto network_dnn = factory->getDNN();
auto network_name = network_dnn->getName();
printf("\nNetwork name: %s\n", network_name.c_str());
```

⁷ The code highlighted in yellow contains a typo in the setting of these configuration variables. These lines should be as follows:

```
environment.setPropertyInt("number_of_cores", 1);
```

Execution

Aside from the initialisation steps outlined previously, the remainder of the application code is identical to standard Klepsydra AI examples. The predict and callback functions are utilised for managing the execution cycles of the AI application, consistent with the general Klepsydra API design.

```
// Random input
auto inputShapeSize = network_dnn->getInputDataShape().size();
std::vector<float> vec_input(inputShapeSize);
for (size_t i = 0; i < inputShapeSize; i++) {
    vec_input[i] = static_cast<float>(i);
}

auto start_ms = std::chrono::high_resolution_clock::now();
// Prepare inference results printing callback

auto printPredictCallback = [&](const unsigned long &id, const std::vector<float> &result) {
    auto end_ms = std::chrono::high_resolution_clock::now();
    auto elapsed_time = std::chrono::duration_cast<std::chrono::milliseconds>(end_ms - start_ms)
        .count();

    printf("Prediction execution time (latency) %lld ms\n", elapsed_time);
    printf("Prediction %lu of size %zu: ", id, result.size());
    for (const auto &value : result) {
        printf("%0.2f, ", value);
    }
    printf("\n");

    ai_data_ready = true;
};

network_dnn->setCallback(printPredictCallback);

// Run inference iterations
for (size_t i = 0; i < TEST_ITERATIONS; i++) {
    start_ms = std::chrono::high_resolution_clock::now();
    network_dnn->predict(vec_input);

    // Wait for previous inference printing callback
    while (!ai_data_ready) {
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }
    ai_data_ready = false;
}

printf("\nExample finished correctly\n");
#ifdef __rtems__
    rtems_task_exit();
#else
    return 0;
#endif
}
```

Performance Benchmarks

Two models were evaluated: the standard MobileNet download and a dummy quantized MobileNet. Both models were tested on the GR740 Mini Rev1 running at 250 MHz using two thread configurations (1 parallel thread and 4 parallel threads).

Notice the high determinism of the solution—the latency variation is practically non-existent, demonstrating its exceptional safety and predictable performance.

The test results are detailed below:

Model	Model Required Memory	1 parallel thread	4 parallel threads
Mobilenet 2-12 float32	14Mb	6'000 ms	1'900 ms
Mobilenet V2 uint8	3.5Mb	9'200 ms	2'800 ms

ANNEX C: Klepsydra AI on the GR740 on Linux

Overview

This annex details the procedure for deploying Artificial Intelligence (AI) applications onto the GR740 processor, utilizing the Klepsydra AI libraries specifically compiled for the Linux environment. The targeted deployment architecture follows a two-step process: first, the Linux image containing the Klepsydra software is built and deployed to the GR740; second, the AI application binary is compiled and copied to the running Linux image (this is an example procedure that can be adapted to your specific workflow):

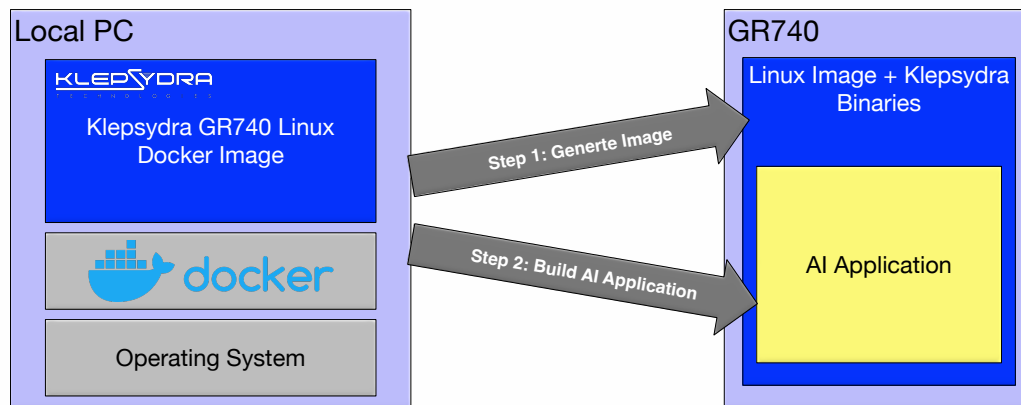


Figure 1: GR740 Linux Deployment Procedure

Prerequisites

This document is focused strictly on the deployment procedure and does not cover the general configuration, initialisation, or full Application Programming Interface (API) of the Klepsydra framework.

Before proceeding with this annex, the user must be familiar with and consult two guides:

- Klepsydra AI Getting Started Guide v25.5.4
- Klepsydra AI ONNX Loader V25.5.4

Deployment procedure

In order to run AI applications on the GR740 / Linux, two things are required: no third party dependencies and no compilation at the target. In order to this, this user guide explains how to do this in the following manner (This procedure serves as a reference implementation. You are free to adapt or automate these steps within your custom build pipeline):

1. Build and Deploy the Base Linux Image: Board setup and environment configuration.
 - a. Generate Image: Build the target Linux image, embedding the core Klepsydra runtime engine and required drivers.
 - b. Flash GR740: Deploy the compiled Linux image to the GR740 board and boot the device.
2. Convert the AI Model to Klepsydra Format. This conversion strips heavy third-party framework dependencies (such as Protobuf) from the target environment, yielding a lightweight engine optimized for embedded execution..
 - a. Export ONNX: Export your trained AI model into standard `.onnx` format from your framework (e.g., PyTorch or TensorFlow).
 - b. Convert Model: Pass the `.onnx` file through the Klepsydra Model Compiler to convert it into Klepsydra's proprietary binary format.
3. Build and Deploy the AI Application: Host compilation and target execution.

- a. Compile Application: Build the host AI application binary against the converted Klepsydra model file and Klepsydra APIs.
 - b. Transfer Binary: Copy the resulting application binary and optimized model file to the running Linux environment on the GR740 via SSH/SCP.
4. Execute: Run the binary on the target board.

Benefits of the solution

This integrated deployment methodology, leveraging Klepsydra AI on the GR740/Linux platform, delivers significant operational and performance advantages:

- Accelerated Deployment and Performance: The solution enables the rapid deployment of AI applications with performance optimized specifically for the GR740 architecture.
- Multicore Execution: The utilization of Linux inherently supports the execution of AI applications across multiple cores, maximizing the available processing power of the GR740.
- Superior Performance Benchmarks: Compared to established alternatives like TensorFlow Lite Micro and Apache TVM, Klepsydra AI consistently demonstrates substantially better processing efficiency on the GR740:
 - Achieving up to 2× faster processing in single-core execution.
 - Delivering 6× to 8× faster processing during multicore execution.

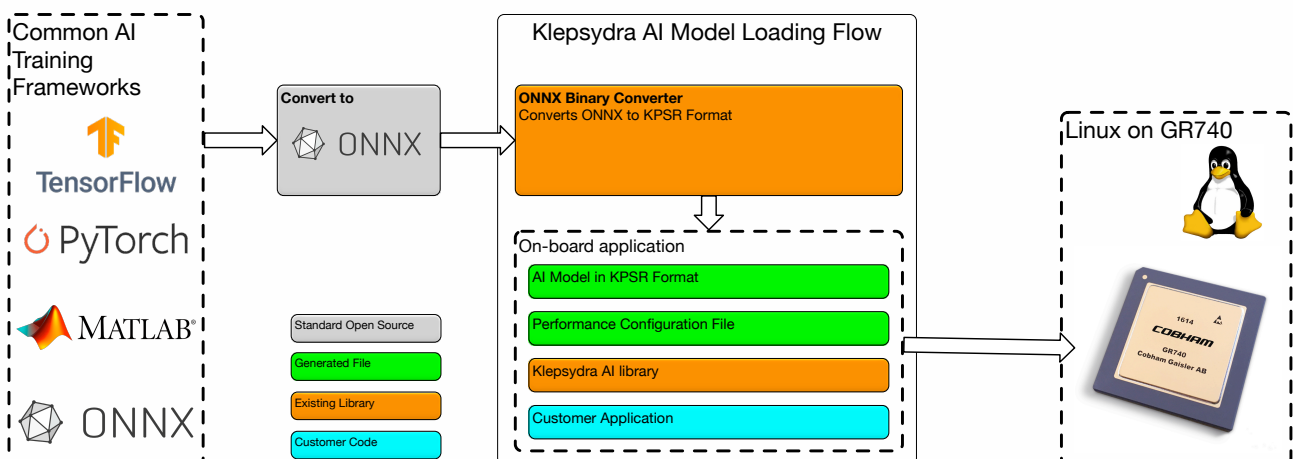


Figure 2: Deployment workflow including the integrated binary with model, AI libraries, application code and configuration files.

Installation package content and requirements

Prerequisites and Requirements

To successfully follow the procedures outlined in this guide, the following elements are required:

Category	Requirement	Notes
Development Host	Linux-based Laptop or PC	With Docker support.
Software	Docker Engine	Required for running the provided development environment image.
Target Access	Connection to GR740 Hardware or TSIM Simulator	Necessary for uploading and executing the final deployable binary.
Knowledge	Makefile, Docker and Linux Command Line	Fundamental understanding of these environments is assumed.

Software Package Content

The provided software delivery consists of a comprehensive Docker image containing all necessary components to streamline the development and deployment process:

- **Klepsydra AI Binaries:** Pre-compiled libraries and tools required for the GR740/Linux target.
- **Reference Examples:** A complete set of examples, including fully functional customer application code, associated Klepsydra configuration files, and a sample AI model.
GR740 Linux Version: Contains AI examples for the "minimal build"—a lightweight setup free of third-party dependencies. Please see the Klepsydra AI Getting Started Guide for further details.
- **Cross-Compilation Toolchain:** The necessary tools for building the deployable binary. Note: The specific version of the SPARC/LEON Linux cross-compilation toolchain may be upgraded or downgraded according to the customer's existing licensing agreement with Gaisler. The main requirements are to have GCC 8 or above, atomic support and RTEMS6 SMP support.
- **Klepsydra AI Docker Binaries and Tools:** A suite of conversion tools for model preparation and cross-platform deployment. It includes utilities to convert ONNX models into the Klepsydra (.kpsr) format, as well as data transformation tools to map numeric formats between x86_64/AArch64 host environments and the GR740 target architecture.

The Klepsydra AI Docker image is structured to provide all necessary binaries, header files, examples, and tools in standard, easily accessible locations within the container environment.

Specifically, the key contents are located as follows:

Directory/File	Content Description
<code>\${GR740_SYSROOT}/usr/local/lib</code>	Binaries (Libraries and executables).
<code>\${GR740_SYSROOT}/usr/local/include</code>	Header Files (C++ interface files).
<code>/usr/local/src/kpe-ai-examples</code>	Example Code (Source files for demonstration projects).
<code>/usr/local/share/doc/kpe-ai/Makefile.sample</code>	A Sample Makefile provided for reference and configuration of build processes.
<code>\${GR740_SYSROOT}</code>	Linux Image root folder
<code>\${GR740_CMAKE}</code>	Cross-Compiler (G++) toolchain file
<code>/usr/local/bin/kpsr_*</code>	Klepsydra AI tools for model conversion and model validation
<code>/usr/local/bin/dnn-stats-example</code>	Model validation tool as well as statistics gatherer

This structure ensures all necessary components for development, cross-compilation, and ONNX model preparation are readily available within the Docker environment.

Example deployment guide

This section provides a step-by-step procedural guide designed to walk the user through the process of compiling, deploying, and executing a complete Klepsydra AI application example on the GR740 target hardware. There are four sections corresponding to the four parts of the process:

- Step 0: Download and installation of the software
- Step 1: Preparation of the Linux Image with Klepsydra AI binaries
- Step 2: Compilation of the AI examples
- Step 3: Deployment and execution

Step 0: Software installation

Download Klepsydra AI software for GR740

Download Klepsydra AI software for GR740

Please NOTE:

The download page is currently being updated and is temporarily unavailable. In the meantime, software packages are being provided directly via S3 links provided via email. Thus please go to [“Start the docker”](#) further below to continue the installation.

Start the docker

Load docker image tarball:

```
$ docker load < kpe-ai-gr740-dev-dev_docker_25.5.5.tar.gz
```

Expected output:

...

```
Loaded image: klepsydra-technologies/kpe-ai-gr740-dev:25.5.5
```

Run GR740 Cross Compilation docker container.

Run container in detached mode⁸:

```
$ docker run -itd --name CONTAINER_NAME klepsydra-technologies/kpe-ai-gr740-dev:25.5.5
```

Check that the container is running:

```
$ docker ps
```

To interact with the running container environment, you must attach to its execution session using either the container name or the container ID:

```
$ docker exec -it CONTAINER_NAME bash
```

```
root@<CONTAINER_ID>:/#
```

To exit from container execution just type “exit”

⁸ Where CONTAINER_NAME is any valid text, e.g., klepsydra_ai_gr740

Step 1: Preparation of the Linux Image with Klepsydra AI binaries

Overview

Environment & Deliverables

Klepsydra provides a Docker container pre-configured with the GR740 cross-compilation toolchain and `kpe-ai` packages, alongside a sample Linux Buildroot image for target execution.

Target Network Configuration

- **Default IP:** The sample Linux Buildroot image is pre-configured with a static IP address of 172.22.10.110 on its Ethernet interface.
- **Modifying Network Settings:** To assign a custom IP address, edit the network configuration file within the Docker container prior to building: `/opt/gaisler-buildroot-2024.02-1.2/output/target/etc/network/interfaces`

Docker Container Environment

- **Linux Image Location:** The sample RAM image is located at `/opt/images/image.ram`.
- **Klepsydra Libraries:** Installed directly in the cross-compilation sysroot at: `/opt/sparc-buildroot-linux-gnu_sdk-buildroot/sparc-buildroot-linux-gnu/sysroot/lib`

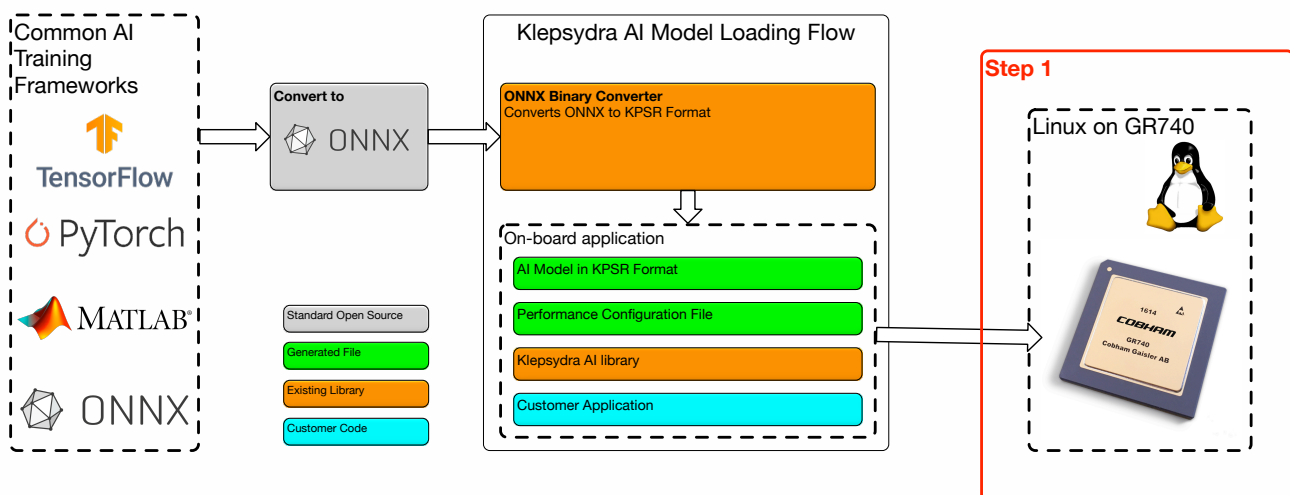


Figure 3: Step 1 Preparation of the Linux Image with Klepsydra AI binaries

Building and Deploying the Linux Image

i. Execute the following commands inside the Docker container:

```
cd /opt/gaisler-buildroot-2024.02-1.2
make -j2
```

ii. The compiled image will be located at:

```
/opt/gaisler-buildroot-2024.02-1.2/output/images/image.ram
```

iii. Use `docker cp` to copy `image.ram` from the container to your host system. E.g.:

```
docker cp CONTAINER_NAME:/opt/gaisler-buildroot-2024.02-1.2/output/images/
image.ram /var/tmp/
```

iv. Load the image onto the GR740 board using GRMON:

```
grmon -e "load image.ram; run"
```

v. Once Linux has booted on the GR740, log in using the default credentials:

- Username: root
- Password: root

Validation of the Linux Image

i. Verify Klepsydra Libraries Installation: Run the following command in your terminal to list the installed Klepsydra shared libraries:

```
ls /usr/local/lib/libkp*
```

Ensure the output confirms the presence of both the SDK and Admin core libraries:

```
/usr/local/lib/libkpe-sdk.so
/usr/local/lib/libkpe-sdk.so.5
/usr/local/lib/libkpe-sdk.so.5.1.0
/usr/local/lib/libkpsr_admin.so
/usr/local/lib/libkpsr_admin.so.17
/usr/local/lib/libkpsr_admin.so.17.0.2
...
```

ii. Verify interface status and check the IP address:

```
ifconfig -a
```

Ensure the network interface displays the assigned static IP (default: 172.22.10.110).

iii. Run from host workstation connected over Ethernet. Verify host-to-target connectivity by connecting from your Ethernet-attached workstation via SSH:

```
ssh root@172.22.10.110
```

When prompted, enter password root.

Step 2: Compilation of the AI examples

Overview

Out of the four included demonstration programs, this section focuses on building and executing two key applications:

- ai-min-example and
- ai-min-raw-images-example.

For complete descriptions of all available examples, please refer to the Klepsydra AI Getting Started Guide.

The deployment workflow follows three main steps:

- a. Cross-compile the binaries within the Docker container environment.
- b. Preparation of the image data for the example ai-min-raw-examples
- c. Packaging

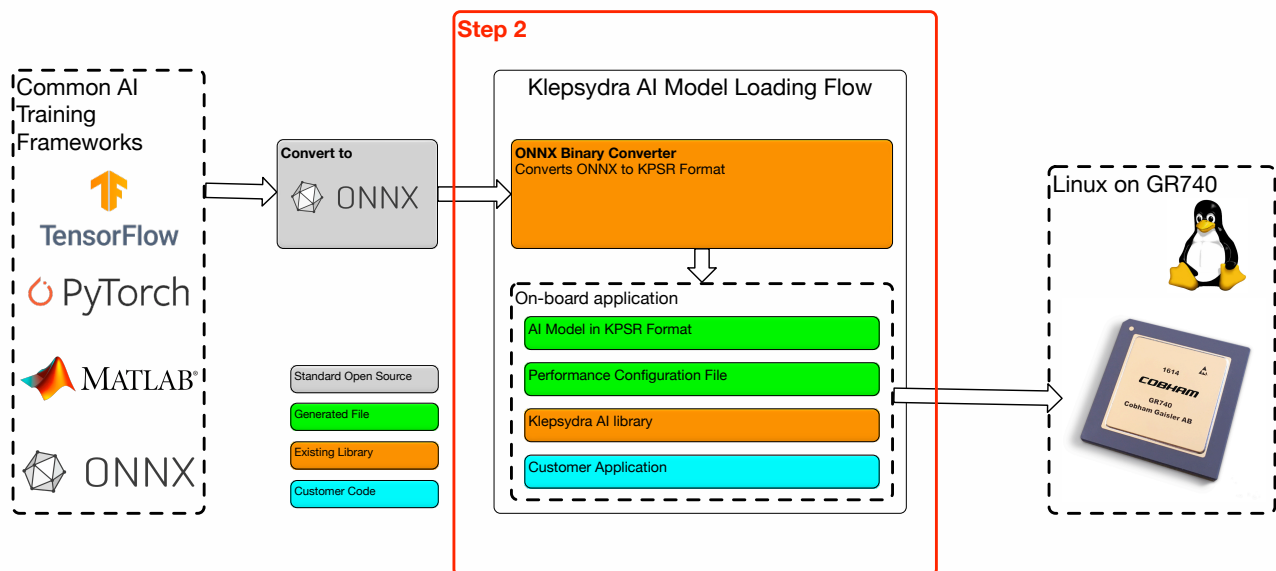


Figure 4: Step 2 Compilation of the AI examples

Cross-compile the binaries within the Docker container environment

Copy the examples to a local folder (e.g., /home/kpsrci)

```
cp -r /usr/local/src/kpe-ai-examples .
```

Go to the ai-min-example folder and create the build folder:

```
cd kpe-ai-examples/ai-min-example/
mkdir build
cd build/
```

Using the provided toolchain file, compile the example:

```
cmake -DCMAKE_TOOLCHAIN_FILE=${GR740_CMAKE} ..
```

make

This should have generated a binary file called `ai-min-example` under the build folder.

Repeat the same process for the `ai-min-raw-images-example`.

Preparation of the image data for the example `ai-min-raw-examples`

The `ai-min-raw-images-example` requires an extra conversion step due to CPU architecture differences:

- Sample raw images are stored using little-endian floating-point format (x86_64/AArch64).
- The GR740 processor operates on a big-endian architecture.

Without converting the binary data, the AI model on the GR740 will fail to classify the images correctly. To resolve this, the included data conversion tool must be compiled in two distinct environments:

- Docker Container (Host): Converts raw `float32` images into an architecture-agnostic text representation (`std::string`).
- GR740 Board (Target): Re-encodes the text files back into big-endian `float32` binaries natively. This is done as part of the compilation step presented in the previous subsection.

- Compile and Run Converter in Docker: Host-side data preparation.

Navigate to the example directory and build the converter tool:

```
cd ai-min-raw-images-example/
mkdir build_host && cd build_host
cmake -DKPSR_CONVERSION_TOOL_ONLY=true ..
make
```

- Convert the source binary float images into text representation:

```
./ai-min-raw-images-example_converter float2str ../images ../images/str/
```

Converter usage reference:

```
Usage: ./ai-min-raw-images-example_converter <operation> <input_dir>
<output_dir> [width=224] [height=224] [channels=3]
```

Operations:

```
float2str - Convert binary float image files to string text files
str2float - Convert string text files back to binary float files
```

Packaging

Compress both folder using tar:

```
cd /home/kpsrci
tar cvf ai-min-example.tar ai-min-example
tar cvf ai-min-raw-images-example.tar ai-min-raw-images-example
```

Step 3: Deployment and execution

The final step is the deployment and execution in the GR740. This process consists of the following commands:

- Copy the compiled example tarballs to the GR740
- Pre-process the images for the ai-min-raw-images-example
- Execution

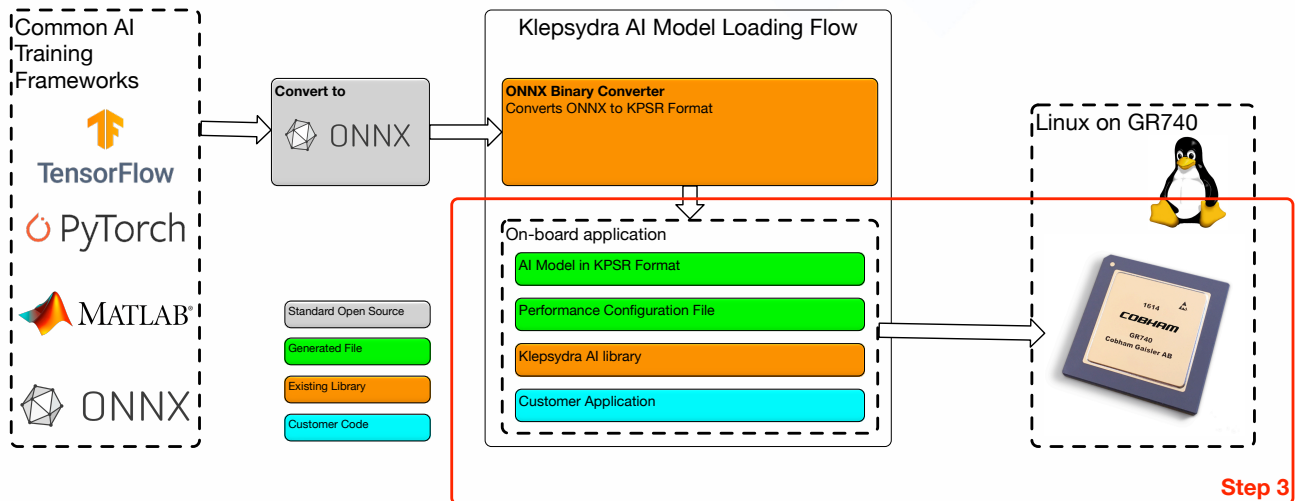


Figure 5: Step 3 Deployment and execution

Copy the compiled example tarballs to the GR740

i. Copy the tarballs out of the docker:

```
docker cp CONTAINER_NAME:/home/kpsrci/ai-min-example.tar /var/tmp
docker cp CONTAINER_NAME:/home/kpsrci/ai-min-raw-images-example.tar /var/tmp
```

ii. Make the tarballs available to the computer connected via Ethernet to the GR740 and copy the examples

```
scp ai-min-*.tar root@172.22.10.110:/root/
```

iii. Expand the files on the GR740

```
tar xvf ai-min-example.tar
tar xvf ai-min-raw-images-example.tar
```

Pre-process the images for the ai-min-raw-images-example

Similar to the previous process, we need to convert the images back to float32 on the GR740:

```
cd ../../ai-min-raw-images-example/build
./ai-min-raw-images-example_converter str2float ../images/str/ ../images/gr740/
```

Rename the images to the original name:

```
cd ../images/gr740
```

```
mv reconstructed_converted_n01514859_hen.raw.txt.bin n01514859_hen.raw
mv reconstructed_converted_n02086079_Pekinese.raw.txt.bin n02086079_Pekinese.raw
mv reconstructed_converted_n03534580_hoopskirt.raw.txt.bin
n03534580_hoopskirt.raw
```

Execution

The final step is the execution of the examples:

- i. Copy the provide trial licence to the default folder

```
mkdir /usr/local/share/licenses
```

And either copy the license file to that location or create the file manually and copy the content from the provided file.

- ii. Execute the ai-min-example

```
cd /root/ai-min-example/build
./ai-min-example -f ../conf/mem_dnn_conf.json -m ../models/alexnet.kpsr -t 6000
```

The expected output should look like this:

```
inputData.configurationFile: ../conf/mem_dnn_conf.json
inputData.modelFile: ../models/alexnet.kpsr
inputData.periodInMS: 6000
[Mon Aug 31 09:44:19 2026] [09:44:19 565142] [klepsydra] [info] [295] Empty parallel_threads_cores, generating one
by default
[Mon Aug 31 09:44:19 2026] [09:44:19 565945] [klepsydra] [info] [295] CPUs for parallel thread affinity [0, 1, 2,
3]
[Mon Aug 31 09:44:19 2026] [09:44:19 579129] [klepsydra] [info] [295] Layers to be optimised out:
StatefulPartitionedCall/functional_1/re_lu/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 583883] [klepsydra] [info] [295] Removing StatefulPartitionedCall/
functional_1/re_lu/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 587692] [klepsydra] [info] [295] Layers to be optimised out:
StatefulPartitionedCall/functional_1/re_lu_1/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 590874] [klepsydra] [info] [295] Removing StatefulPartitionedCall/
functional_1/re_lu_1/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 591864] [klepsydra] [info] [295] Layers to be optimised out:
StatefulPartitionedCall/functional_1/re_lu_2/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 592593] [klepsydra] [info] [295] Removing StatefulPartitionedCall/
functional_1/re_lu_2/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 593090] [klepsydra] [info] [295] Layers to be optimised out:
StatefulPartitionedCall/functional_1/re_lu_3/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 593949] [klepsydra] [info] [295] Removing StatefulPartitionedCall/
functional_1/re_lu_3/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 594451] [klepsydra] [info] [295] Layers to be optimised out:
StatefulPartitionedCall/functional_1/re_lu_4/Relu
[Mon Aug 31 09:44:19 2026] [09:44:19 594973] [klepsydra] [info] [295] Removing StatefulPartitionedCall/
functional_1/re_lu_4/Relu
[Mon Aug 31 09:44:21 2026] [09:44:21 031555] [klepsydra] [info] [295] Calling startFactories void
kpsr::streaming::SmartPublishSubscribeFactory::startFactories()
[Mon Aug 31 09:44:21 2026] [09:44:21 038588] [klepsydra] [info] [295] Calling startFactories void
kpsr::streaming::SmartPublishSubscribeFactory::startFactories()
[Mon Aug 31 09:44:21 2026] [09:44:21 039749] [klepsydra] [info] [295] Calling startFactories void
kpsr::streaming::SmartPublishSubscribeFactory::startFactories()
[Mon Aug 31 09:44:21 2026] [09:44:21 040900] [klepsydra] [info] [295] Calling startFactories void
kpsr::streaming::SmartPublishSubscribeFactory::startFactories()
[Mon Aug 31 09:44:21 2026] [09:44:21 042049] [klepsydra] [info] [295] Calling startFactories void
kpsr::streaming::SmartPublishSubscribeFactory::startFactories()
[Mon Aug 31 09:44:21 2026] [09:44:21 043196] [klepsydra] [info] [295] Calling startFactories void
kpsr::streaming::SmartPublishSubscribeFactory::startFactories()
It took 2581ms to load the model.
Applying AI to dummy image 0 of 100
Image : 0 assigned id : 0
```

```

Use this id in callback to match input to result.
Got result for input with id: 0
Processed 0 of 100 in 5650 ms
Applying AI to dummy image 1 of 100
Image : 1 assigned id : 1
Use this id in callback to match input to result.
Got result for input with id: 1
Processed 1 of 100 in 5694 ms
Applying AI to dummy image 2 of 100
Image : 2 assigned id : 2
Use this id in callback to match input to result.

```

Important note:

The -t parameter defines the inference publishing period in milliseconds (e.g., -t 6000 for 6000 ms). For this specific model on the GR740, the period must not be set lower than 6000 ms.

Using a lower interval—or leaving it at the default 100ms—will overload the target system, triggering memory pool errors and severe latency spikes. Refer to the Klepsydra AI Getting Started Guide for further details.

iii. Execute the ai-min-raw-images-example

```

cd /root/ai-min-raw-images-example/build
./ai-min-raw-images-example -f ../conf/mem_dnn_conf.json -m ../models/
mobilenetv2-7.kpsr -d ../images/gr740/ -t 3000

```

Please see the note above about the time interval parameter.

The expected output should like the text below, proving that model is classifying the images correctly:

```

Read a list of 3 images in folder: ../images/gr740/
Applying AI to image n01514859_hen.raw
Image 0 with name: "n01514859_hen.raw" is classified as hen with prob 0.731943
Processed 1 of 3
Applying AI to image n02086079_Pekinese.raw
Image 1 with name: "n02086079_Pekinese.raw" is classified as Pekinese, Pekingese, Peke
with prob 0.980942
Processed 2 of 3
Applying AI to image n03534580_hoopskirt.raw
Image 2 with name: "n03534580_hoopskirt.raw" is classified as hoopskirt, crinoline with
prob 0.853859
Processed 3 of 3
Accurate predictions: 3 of 3

```

Troubleshooting

Licensing issues

If the following error is shown during the examples execution:

```

inputData.configurationFile: ../conf/mem_dnn_conf.json
inputData.modelFile: ../models/alexnet.kpsr
inputData.periodInMS: 6000
[Thu Jan 1 00:39:32 1970] [00:39:32 766343] [klepsydra] [error] [160] License Error: license file not found
[Thu Jan 1 00:39:32 1970] [00:39:32 766758] [klepsydra] [error] [160] Please provide the path to the license file
in
[Thu Jan 1 00:39:32 1970] [00:39:32 766907] [klepsydra] [error] [160] the KPSR_LICENSE_FILE environment variable.
[Thu Jan 1 00:39:32 1970] [00:39:32 767078] [klepsydra] [error] [160] If the environment variable is not provided,
/usr/local/share/licenses/klepsydra.lic will be used.
[Thu Jan 1 00:39:32 1970] [00:39:32 771056] [klepsydra] [error] [160] -----
[Thu Jan 1 00:39:32 1970] [00:39:32 771224] [klepsydra] [error] [160] PC Signature: AAAA-AHzM-AUY=
[Thu Jan 1 00:39:32 1970] [00:39:32 771372] [klepsydra] [error] [160] -----

```

There might be two reasons for that:

- Make sure that the license file content is correct in the exact location shown in the error (/usr/local/share/licenses/klepsydra.lic)
- Make sure that the system date is correct. It might happen that after booting, the start date remain at 01.01.1970. In this case, run the following command from the computer connected to the GR740:

```
ssh root@172.22.10.110 "date -s '$(date -u +"%Y-%m-%d %H:%M:%S")'"
```

And then check the date in the GR740:

```
# date
Mon Aug 31 09:49:12 UTC 2026
```

Connectivity issues

Because the MAC address of the GR740 might change after a reboot / restart of GRMON, when doing scp or ssh, the following error might appear:

```

@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@  WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!  @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@

IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!

Someone could be eavesdropping on you right now (man-in-the-middle attack)!

It is also possible that a host key has just been changed.

The fingerprint for the ED25519 key sent by the remote host is
SHA256:DUU41VOjUNECVIAWtmQ/BL7cJs43JoHX9Q24k01HzQo.
Please contact your system administrator.
Add correct host key in /home/klepsydra/.ssh/known_hosts to get rid of this message.
Offending ECDSA key in /home/klepsydra/.ssh/known_hosts:3
  remove with:
  ssh-keygen -f '/home/klepsydra/.ssh/known_hosts' -R '172.22.10.110'
Host key for 172.22.10.110 has changed and you have requested strict checking.
Host key verification failed.
scp: Connection closed

```

In this case, just executed the provided command in the error message (highlighted)

```
ssh-keygen -f '/home/klepsydra/.ssh/known_hosts' -R '172.22.10.110'
```

After that the connection should work.

About Klepsydra

Overview

Founded in Switzerland in 2018, Klepsydra specializes in ultra-high-performance edge AI and data processing software for space, aerospace, smart mobility, and IoT applications. Led by CEO Pablo Ghiglino and an advanced engineering team of PhDs and academic experts, Klepsydra provides the fastest AI runtime engine available for embedded edge hardware.

Klepsydra AI stands as the market's only solution pre-qualified for ECSS flight software standards, reflecting extensive engineering partnerships with ESA (European Space Agency) and premier aerospace companies. Combined with maximum cybersecurity protections and deterministic execution, Klepsydra enables space missions to deploy high-throughput AI workloads with complete mission confidence.

Contact

Klepsydra Technologies AG

Zug, 6300

www.klepsydra.com

sales@klepsydra.com